P0891R1
Everyone Deserves a Little Order

Published Proposal, 2018-10-27

This version:
https://github.com/atomgalaxy/a-little-order/strong-ordering.bs

Author:
Gašper Ažman <gasper.azman@gmail.com>

Audience:
LEWG, LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The specification of ordering algorithms at the end of [P0768R1] does not provide the ability to provide a default order for user-defined types (since they are specified in such a way that they are not intended to be customization points), and yet mixes in such a customization for iec559 floating point types. This paper suggests splitting that capability out into a separate customization point.

Table of Contents

§ 1. Revision History

- r1: Incorporated feedback from EWG meeting in Rappersville.

The feedback was:

- Remove the floating point exception (bullet 1.1) as R0 recommended, since Unicode strings, etc., are a possible rationale.
- Do not propose propose making existing `*_order` functions "customization points" (as used in [P0551R3]).
- Add a new `default_order` customization point, along with a bikeshedding section on its actual name, with the behaviour:

    - It has the IEC 559 behavior from bullet 1.1 of `strong_order`
    - It is defined for all (other) floating-point types; it is implementation-defined whether it is consistent with the partial order from the comparison operators. (Implementations should do this.)
    - It is a customization point (à la [P0551R3]).
- Investigate the possibility of adding Lawrence's weak order (from [P0100R2]) for floating-point numbers (which did not make it in with spaceship).

§ 2. Status of this paper

This paper has been seen by LEWG, provided feedback, and reworked. It is ready to be seen again by LEWG.

§ 3. Problem Description

This paper is a proposal to amend a library extension that has been voted into the working draft as part of [P0768R1].

This paper proposes a new ordering customization point for possibly non-semantic strong orderings, which *should* be equal to `std::strong_order` for strongly-ordered types, and can be customized to represent an arbitrary total order for types that do not have a semantic ordering.

The current C++ standard does not have an explicitly designated customization point for providing a *default ordering*. *Elements of Programming* uses `less<T>::operator()` for this purpose, as does the global order for pointers; but with the introduction of `operator<=>`, `less<T>` is missing features, such as computing equality without calling it twice. It has also failed to get adoption for this purpose throughout the years, perhaps exactly due to the missing features.

> Note: see §4 Exposition: On Natural and Default Orderings for the definitions and discussion of orderings.

The wording of point 1.1 of the `std::strong_order` algorithm suggests that `std::strong_order` is finally this missing customization point for specifying a default ordering for types whose natural ordering is not strong and total, since it does exactly that for the iec559 types.

The issue is that the rest of the points make this function rather unsuitable for use as a customization point, since the language explicitly makes it not SFINAE-friendly. In the event that it cannot be

synthesized, it is marked as *deleted*, and not as *"shall not participate in overload resolution"*.

LEWG has expressed a strong preference to introducing a new customization point for such non-semantic strong orderings, and clarified that `std::strong_order` should not be a customization point.

§ 4. Exposition: On Natural and Default Orderings

There are obviously many reasons for sorting. However, this paper is chiefly concerned with the division between the *natural ordering* and the *default total ordering* as required for **Regular** types by Stepanov and McJones in their seminal work *Elements of Programming* (page 62, section 4.4).

The **natural ordering** is the ordering that makes semantic sense for a type. This is the ordering that `operator<=>` and its library extensions are tailor-made for: not every type is ordered (or even equality-comparable), and when a type supports an ordering, it might be strong, partial, or weak.

We use these orderings when we need them to make sense - heaps, scheduling tasks by topological sorts, various displays for users, etc. Not all value types have a natural ordering, because not all types are ordered. The Gaussian integers are one such type.

The **default ordering**, from *Elements of Programming* is the finest ordering (transitive antisymmetric antireflexive relation) that a type admits, with its equality is defined by value-substitutability (unequal elements must be ordered); it is always strong and total, and might not make semantic sense.

According to *Elements of Programming*, every **Regular** type should provide a default ordering.

A type with a default ordering is far more useful than one without; ordering enables the use of tree-based containers (i.e. `map`, `set`), and algorithms based on sorted data (`unique`, the various set algorithms, `merge`, and the various versions of binary search) -- and this is just the tip of the iceberg. The only requirement for the above is having *a* total strong ordering - what the ordering *means* is utterly irrelevant, we only require a global order relation.

The lexicographic ordering of the Gaussian integers is a good example of a default ordering.

Another excellent example is `float` -- its various `NaN`s and infinities are not ordered, which is the reason its natural ordering is not suitable as a default ordering. However, iec559 defines a total strong ordering for those values, thus enabling the uses outlined above.

§ 5. Status Quo

For reference, the current specification for the `std::strong_order` algorithm is as follows:

```
template<class T>
constexpr strong_ordering strong_order(const T& a, const T& b);
```

1. Effects: Compares two values and produces a result of type `strong_ordering`:
    1. If `numeric_limits<T>::is_iec559` is true, returns a result of type `strong_ordering` that is consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60559.
    2. Otherwise, returns `a <=> b` if that expression is well-formed and convertible to `strong_ordering`.
    3. Otherwise, if the expression `a <=> b` is well-formed, then the function shall be defined as deleted.
    4. Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, returns `strong_ordering::equal` when `a == b` is `true`, otherwise returns `strong_ordering::less` when `a < b` is `true`, and otherwise returns `strong_ordering::greater`.
    5. Otherwise, the function shall be defined as deleted.

§ 6. Proposal

§ 6.1. Introduce a new customization point for an arbitrary total ordering

Introduce a new algorithm, with the name `std::default_order` (but see §7 Name of The Default Order Algorithm), with the following semantics:

Let `T` be a type, and `a` and `b` be values of type `T` (as if `a` and `b` were `declval<T>()`):

1. If well-formed, the expression `default_order(a, b)` shall return an object convertible to `strong_ordering`.
2. `default_order` shall be a designated customization point in the sense of [P0551R3].
3. If the expression `strong_order(a, b)` is well-formed, then the expression `default_order(a, b)` is well-formed.

4. If not explicitly specialized for a type `T` and `strong_order(a, b)` is well-formed, the expression `default_order(a, b)` shall return the result of the expression `strong_order(a, b)`. (Note: this means that `default_order` defaults to `strong_order` if available for the type. -- end note.)

§ 6.2. Move the iec559 treatment (point 1.1) from `strong_order` to `default_order`

Since this paper adds an explicit customization point for a non-semantic total order on any type, the exception for iec559 floating-point types can now be implemented by explicitly providing an implementation for the `default_order` customization point for those types.

The concrete proposal is as follows:

1. remove point 1.1 of the `strong_order` algorithm, which removes the exception for iec559 types.
2. add a provision to `default_order`
    1. if `(bool) numeric_limits<T>::is_iec559` is `true`, the expression `default_order(a, b)` returns a result of type `strong_ordering` that is consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60559.

Remark: libraries are encouraged to provide implementations of this customization point for their user-defined types, especially if the `operator<=>` for the type does not provide a `strong_ordering`, to enable possibly non-semantic total orderings over their entire domain. The use of implementations of this customization point that do not define a strict total order render the program ill-formed (no diagnostic required).

§ 7. Name of The Default Order Algorithm

The new customization point that exposes an arbitrary order for any type that cares to provide one needs a name. This paper suggests a 5-way poll to LEWG on the following options:

§ 7.1. `default_order`

- Pros:
    - it is what *Elements of Programmming* calls it, a-priori giving it wide recognition

- it is reasonably short
- the name is semantically netural.
- does not imply that the order has any meaning past *strong* and *total* (eg. "lexicographic")

- Cons:
  - it implies neither that it is total, nor that it is strong, despite the requirement it be both.

`default_order` is the favorite of the paper author, as well as most of the reviewers of this paper.

§ 7.2. `total_order`

- Pros:
  - communicates the totality of the order
  - `totalOrder` is the name `ISO/IEC/IEEE 60559` chose for the order over floating point types that implement these semantics
  - it is reasonably short
- Cons:
  - does not imply it is a strong order
  - at least to the author's mind, vaguely implies semantics past *strong* and *total*, which might not be the case.

§ 7.3. `arbitrary_order`

- Pros:
  - clearly implies it might not be anything past *strong* and *total*
- Cons:
  - implies neither that it is total, nor that it is strong
  - very vague
  - sounds slightly derogatory for a facility that is to be used mostly by critical algorithms and data structuers

§ 7.4. `default_strong_order`

- Pros:
  - clear
  - roughly in line with *Elements of Programming*
- Cons:
  - very long for a facitlity that will be used for in-line calls to `sort` and `unique`
  - still not clear it is total

§ 7.5. `default_total_order`

- Pros:
  - clear
  - roughly in line with iec559 and *Elements of Programming*
- Cons:
  - very long for a facitlity that will be used for in-line calls to `sort` and `unique`
  - still not clear it is strong

§ 8. On Compatibility Between the Natural and Default Orderings

Elements of Programming specifies that for types where the natural and default orderings differ, the default ordering should be *finer* than the natural one: that is, if `a` and `b` are *comparable* and compare unequal under `<=>`, the default order produces the same result (less or greater).

It is the opinion of the author that requiring this in the language of the standard library as a mandatory semantic constraint seems like a bad idea.

For instance, if one takes the Gaussian integers ordered by the Manhattan distance to zero (sum of absolute values of the two components), the compatible total order (a lexicographic ordering of every equivalence class) is far slower to compute than the simple lexicographic one.

Furthermore, if needed, a *finer* compatible total order can always be achieved on the fly by comparing with the natural order first: if the result is `less` or `greater`, keep the result; otherwise, fall back on the default ordering.

§ 9. Why not just make `strong_order` a customization point?

Main reasons:

1. it would inhibit providing both a natural (`<=>`, `strong_order`) (and possibly slow) and a default (fast) order for a type
2. the committee guidance strongly preferred this option, as it keeps the meaning of `strong_order` fixed (since it is not a customization point)
3. it is less surprising if the order algorithms that are related to order types by name (`weak_order` - `weak_ordering`, `partial_order` - `partial_ordering`, `strong_order` - `strong_ordering`) had the same specification, while a fourth customization point that isn't related by any of them by name serves as the customization point for default order.

It is notable that this was the direction suggested by the orginal paper, but the committee rejected it.

§ 10. Proposed Wording

From section 24.x.4, Comparison Algorithms [cmp.alg], under `strong_order`:

~~(1.1) If `numeric_limits<T>::is_iec559` is true, returns a result of type `strong_ordering` that is consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60559.~~

~~(1.2) Otherwise, returns~~

(1.2) Returns

After 24.x.4 paragraph 3, `weak_order`, add:

```
template<class T> constexpr strong_ordering default_order(const T& a, const T& b);
```

4. *Effects:*

(4.1) if `std::numeric_limits<T>::is_iec559` is true, returns a result of type `strong_ordering` that is consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60559.

(4.2) Otherwise, returns `strong_order(a, b)` if that expression is well-formed and convertible to `strong_ordering`

(4.3) Otherwise, this function shall not participate in overload resolution.

5. *Remarks*: this function is a designated customization point ([namespace.std]).

Under `[cmp.syn]` -> `[cmp.alg]`, insert declaration:

```
template <class T> constexpr strong_ordering default_order(const T& a,
const T& b);
```

§ 11. Acknowledgments

I would like to thank

- **Roger Orr** for bringing this to my attention;
- **Thomas Köppe** for his valuable comments, review, and most of all some extremely clear and laconic wording;
- **Sam Finch** for *thoroughly* breaking my examples, some example code, great substantive comments, and pointing out that the current definition actually breaks types that define a partially-ordered set of comparison operators;
- **Richard Smith** for further fixing my example in light of Concepts, and example code.
- **Herb Sutter and Walter Brown** for providing guidance on customization points.
- **Louis Dionne** for great comments on the structure of the paper and how to bring the focus where it needs to be;
- **Walter Brown** for representing the paper at committee meetings when I could not make it in person, and guidance with direction;
- **Herb Sutter** for his comments and support for getting ordering right.

And, *again*, a special thank-you to Walter Brown, who, with his final lightning talk in Bellevue, reminded me to remember whose shoulders I'm standing on.

Thank you all!

§ Appendix A: Proof `strong_order` is not a valid customization point

Say we have a template struct representing the Gaussian integers, with a *natural order* defined by the Manhattan distance from `0+0i`. This struct still defines a `std::strong_order` to model **Regular**.

> Note: The **Regular** above refers to the *Elements of Programming* concept, not the ISO C++ **Regular**, which is weaker.

> Note: There is no natural order on Gaussian integers, but humor this example, please.

```
namespace user {
  template <typename T>
  struct gaussian {
    static_assert(std::is_integral_v<T>);
    T re;
    T im;

    constexpr std::strong_equality operator==(gassian const& other) const {
      return re == other.re && im == other.im;
    }
    constexpr std::weak_ordering operator<=>(gaussian const& other) const {
      return (*this == other) ? std::weak_ordering::equal
                              : (abs(*this) == abs(other)) ? std::weak_orde
                                                           : abs(*this) <=>
    }
    friend constexpr T abs(gaussian const&) {
      using std::abs;
      return abs(re) + abs(im);
    }

    friend constexpr std::strong_ordering strong_order(gaussian const& x,
                                                       gaussian const& y) {
      // compare lexicographically
      return std::tie(x.re, x.im) <=> std::tie(y.re, y.im);
    }
  };
}
```

Consider a transparent ordering operator for `map`:

```
struct strong_less
  template <typename T, typename U>
  bool operator()(T const& x, U const& y) {
    using std::strong_order;  // use ADL
    return strong_order(x, y) < 0;
  }
  using is_transparent = std::true_type;
};
```

Also say we had a type with an implicit conversion to our `gaussian`:

```
template <typename T>
struct lazy {
  std::function<T()> make;
  operator T() const { return make(); }
};
```

This function now fails to compile, because the chosen `std::strong_order` is deleted.

```
bool exists(lazy<gaussian<int>> const& x,
            std::set<gaussian<int>, strong_less> const& in) {
  /* imagine this being a template in both parameters - it's pretty normal
  return in.count(x);
}
```

The std-provided `std::strong_order` is deleted because it cannot be synthesized from `gaussian`'s `operator<=>`. The reason it is chosen over the friend function, however, is because the standard template matches better than the friend which would require an implicit conversion.

If the std-provided `std::strong_order` did not participate in overload resolution, however, this example would work just fine.

§ References

§ Normative References

[P0551R3]
Walter E. Brown. Thou Shalt Not Specialize std Function Templates!. 16 March 2018. URL: https://wg21.link/p0551r3

§ Informative References

[P0100R2]
Lawrence Crowl. Comparison in C++. 27 November 2016. URL: https://wg21.link/p0100r2

[P0768R1]

Walter E. Brown. Library Support for the Spaceship (Comparison) Operator. 10 November 2017. URL: https://wg21.link/p0768r1